Module 05: Database, Migrations & Eloquent ORM

Duration: 7-10 days | **Level:** Beginner-Intermediate

Migrations

Migrations are version-controlled database schema changes.

```
php artisan make:migration create_tasks_table
php artisan migrate
php artisan migrate:rollback
php artisan migrate:fresh --seed
```

TaskForge Migration Example

```
Schema::create('tasks', function (Blueprint $table) {
$table->id();
$table->foreignId('project_id')->constrained()->cascadeOnDelete();
$table->foreignId('user_id')->constrained()->cascadeOnDelete();
$table->string('title');
$table->text('description')->nullable();
$table->string('status')->default('pending');
$table->string('priority')->default('medium');
$table->timestamp('due_at')->nullable();
$table->timestamps();

$table->index(['status', 'due_at']);
});
```

Column Types Reference

Method	SQL Type
$table->string('name')	VARCHAR(255)
$table->text('body')	TEXT
$table->integer('count')	INT
$table->boolean('active')	BOOLEAN
$table->json('meta')	JSON
$table->foreignId('user_id')	UNSIGNED BIGINT
$table->timestamps()	created_at, updated_at

Eloquent Models

```
class Task extends Model
{
protected $fillable = ['title', 'project_id', 'status'];

protected function casts(): array
{
return [
'status' => TaskStatus::class,  // PHP Enum
'due_at' => 'datetime',
];
}
}
```

CRUD Operations

```
// Create
Task::create(['title' => 'Fix bug', 'project_id' => 1]);

// Read
$task = Task::find(1);
$task = Task::findOrFail(1);  // 404 if missing

// Update
$task->update(['status' => 'completed']);

// Delete
$task->delete();
```

Relationships

One-to-Many

```
// Project has many Tasks
class Project extends Model {
public function tasks(): HasMany {
return $this->hasMany(Task::class);
}
}

// Task belongs to Project
class Task extends Model {
public function project(): BelongsTo {
return $this->belongsTo(Project::class);
}
}
```

Many-to-Many (Assignees)

```
// Task.php
public function assignees(): BelongsToMany {
return $this->belongsToMany(User::class)->withTimestamps();
}

// Attach / sync
$task->assignees()->attach($userId);
$task->assignees()->sync([1, 2, 3]);
```

Eager Loading (N+1 Prevention)

```
// BAD: N+1 queries
$tasks = Task::all();
foreach ($tasks as $task) {
echo $task->project->name;  // Query per task!
}

// GOOD: 2 queries total
$tasks = Task::with(['project', 'assignees'])->get();
```

Query Scopes

```
// Model
public function scopeOverdue($query) {
return $query->where('due_at', '<', now())
```

```
->whereNotIn('status', ['completed', 'cancelled']);
}

// Usage
Task::overdue()->count();
```

Factories & Seeders

Factory - generate fake data:

```
Task::factory()->count(10)->create();
Task::factory()->overdue()->create();
```

Seeder - populate database:

```
php artisan db:seed
php artisan migrate:fresh --seed
```

```
TaskForge seeder creates admin, member, 2 projects, 6 tasks with comments.
```

Exercises

1. Add a tags table with many-to-many relationship to tasks.
2. Write a scope scopeDueThisWeek($query).
3. Use Tinker to query all overdue tasks with projects loaded.

Next Module

-> Module 06: Authentication & Authorization